



Control Flow & Pattern Matching

Python Mastery — Part 1: Foundations · Week 4

Rathinam Trainers & Consultants

Live on Microsoft Teams · recorded

Recap & today

Where we are in the journey

Last week — Strings & Text

- ▶ Indexing & slicing: `s[0]` , `s[-1]` , `s[1:4]` , `s[::-1]`
- ▶ Methods that **return new strings**: `.strip` , `.split` , `.join` , `.lower`
- ▶ **f-strings** + the format mini-language: `f"{price:>8.2f}"`
- ▶ A peek at **t-strings** (PEP 750) and `bytes` vs `str`

You can now build and shape text. Today we make code **decide** and **repeat**.

Today's goal

By the end of this session you'll understand how to:

- ▶ Branch with `if` / `elif` / `else` and reason about **truthiness**
- ▶ Loop with `for`, `while`, and `range()`
- ▶ Steer loops: `break`, `continue`, `pass`, and loop-`else`
- ▶ Dispatch on shape and value with `match` / `case`

Making decisions

`if` / `elif` / `else` and truthiness

Branching with `if` / `elif` / `else`

```
if score >= 90:  
    grade = "A"  
elif score >= 80:  
    grade = "B"  
else:  
    grade = "C"
```

- ▶ **Indentation** defines the block — no braces, no `end`
- ▶ Zero or more `elif`; `else` optional
- ▶ The **first** true branch wins; the rest are skipped

Truthiness — what counts as "true"?

These are the **falsey** values; everything else is truthy:

0 0.0 "" [] {} () set() None False

```
name = ""
if name:           # empty string is falsy
    print("hi", name)
else:
    print("no name given")
```

A value used directly in `if` is tested for truthiness.

Boolean logic & chained comparisons

```
if age >= 18 and has_ticket:  
    enter()  
  
if not done:  
    keep_going()  
  
if 0 <= hour < 24:           # chained – reads like maths  
    valid = True
```

- ▶ and, or, not combine conditions
- ▶ `0 <= hour < 24` means `0 <= hour and hour < 24`

Repeating work

Loops & loop control

for loops & range()

```
for ch in "abc":           # over a sequence
    print(ch)

for i in range(5):         # 0, 1, 2, 3, 4
    print(i)

range(2, 10)               # start, stop
range(0, 10, 2)            # start, stop, step
```

`range(stop)` ends **before** stop. A `for` walks any sequence.

while loops & sentinels

```
count = 3
while count > 0:
    print(count)
    count -= 1          # must move toward the exit!

while True:
    line = input("> ")
    if line == "quit":  # sentinel condition
        break
```

- ▶ `for` = "do this N times / for each item"
- ▶ `while` = "keep going **until** a condition flips"

break, continue, pass

```
for n in nums:
    if n < 0:
        continue          # skip this one, keep looping
    if n == stop:
        break             # leave the loop now
    process(n)

if not ready:
    pass                  # placeholder: do nothing (yet)
```

- ▶ **continue** → next iteration · **break** → exit the loop
- ▶ **pass** → a no-op where syntax needs a statement

The loop-`else` clause

```
for item in inventory:
    if item == target:
        print("found it")
        break
else:
    print("not found")    # runs only if NO break happened
```

- ▶ `else` on a loop fires **when the loop finishes without `break`**
- ▶ Perfect for "search and report not-found"

Let's run every loop, live.

Watch this — Demo 7

Loops & `range` patterns

- ▶ `for i in range(5)` and `for ch in "abc"`
- ▶ A `while` countdown with a **sentinel**
- ▶ `continue` to skip evens, `break` to stop early
- ▶ A `for ... else` that prints "not found" only when no `break`

Loops in action

```
>>> for i in range(0, 6, 2):  
...     print(i)  
0  
2  
4  
  
>>> for n in range(10):  
...     if n % 2 == 0:  
...         continue  
...     if n > 5:  
...         break  
...     print(n)  
1  
3  
5
```

Search with loop-else

```
>>> target = 7
>>> for n in [1, 3, 5]:
...     if n == target:
...         print("found")
...         break
...     else:
...         print("not found")
not found
```

No `break` fired → the `else` ran.

What you just saw

- ▶ `range()` drives counted loops; `for` walks any sequence
- ▶ A `while` repeats **until** its condition flips
- ▶ `continue` skips one turn; `break` leaves entirely
- ▶ `loop-else` runs **only when no** `break` happened

Pattern matching

`match` / `case` — branch on shape

Why `match` / `case`?

- ▶ A cleaner way to branch when the data has **structure**
- ▶ Not just "is it equal?" — also "**what shape is it?**"
- ▶ Stable since 3.10, unchanged in 3.14
- ▶ It can **destructure** and **bind** names in one step

```
match status:  
    case 400: ...  
    case 404: ...  
    case _:    ...           # the wildcard: matches anything
```

Literal, capture & wildcard

```
match command:
    case "quit":                # literal – exact value
        stop()
    case other:                 # capture – binds the value
        print("unknown:", other)
    case _:                     # wildcard – matches, binds nothing
        pass
```

- ▶ A bare name is a **capture**, not a comparison
- ▶ `_` matches anything and discards it

Sequence & or-patterns

```
match command.split():  
    case ["add", task]:           # exactly 2 items  
        add(task)  
    case ["list"] | ["ls"]:      # or-pattern  
        show()  
    case ["tags", *rest]:        # *rest captures the tail  
        tag(rest)
```

- ▶ `[x, y]` matches a 2-element sequence, binding `x`, `y`
- ▶ `*rest` soaks up the remaining items
- ▶ `|` lets one `case` match several shapes

Mapping patterns & guards

```
match payload:
    case {"cmd": c}:                # has a "cmd" key
        run(c)
    case ["done", id] if id.isdigit(): # guard
        complete(int(id))
    case _:
        print("unknown")
```

- ▶ `{"cmd": c}` matches a dict with that key (extras ignored)
- ▶ A **guard** `if ...` adds a condition the pattern must also pass

The order rule

```
match value:  
    case 0:      ...    # checked first  
    case int():  ...    # then this  
    case _:      ...    # always last – the catch-all
```

- ▶ Cases are tried **top to bottom**; first match wins
- ▶ Put **specific** patterns above **general** ones
- ▶ A `case _:` at the end means "no input goes unhandled"

Let's route a command by its shape.

Watch this — Demo 8

`match / case` command dispatcher

- ▶ `match command.split():`
- ▶ `case ["add", task]: · case ["done", id] if id.isdigit():`
- ▶ `case ["list"] | ["ls"]: · case {"cmd": c}: · case _:`
- ▶ Run several inputs live and watch each route

The dispatcher

```
def dispatch(command):  
    match command.split():  
        case ["add", *words]:  
            return f"added: {' '.join(words)}"  
        case ["done", id] if id.isdigit():  
            return f"completed #{id}"  
        case ["list"] | ["ls"]:  
            return "showing all tasks"  
        case []:  
            return "type a command"  
        case _:  
            return "unknown command"
```

One input, the right route

```
>>> dispatch("add buy milk")  
'added: buy milk'  
>>> dispatch("done 7")  
'completed #7'  
>>> dispatch("ls")  
'showing all tasks'  
>>> dispatch("frobnicate")  
'unknown command'
```

The capstone's menu reuses exactly this shape.

What you just saw

- ▶ One `match` routed **five different command shapes**
- ▶ Literal, capture, sequence, or-, and mapping patterns + a guard
- ▶ `*words` captured a variable-length tail
- ▶ The final `case _:` caught everything unexpected

Wrap-up

Q&A + this week's homework

Lab 4 — game + menu

1. **Number-guessing game** — a `while` loop, `break` on a correct guess, and a loop-`else` for "out of tries"
2. `menu.py` — read a command line and route it with `match / case` to printed actions
3. Handle at least **four command shapes** plus an unknown fallback
4. **Submit:** both scripts + sample runs to the Teams channel

Mirror **Demo 7** for the game, **Demo 8** for the menu.

Next week

- ▶ **Week 5 — Functions, Scope & Type Hints**
- ▶ We wrap today's decisions and loops into **reusable functions**
- ▶ The `match`-based menu becomes a typed, documented function

See you in the lab.

Questions?